EC NB-IoT PMU Development Guidebook

EigenCOMM Wireless Microcontroller

Document Description

This document describes the PMU(Power Management Unit) of the NB-IoT chip EC616 / EC617 in detail, and guides users to develop low power applications by using related APIs.

Contents

1. Introduction

1.1 Purpose of the document

This document describes in detail the PMU capability of NB-IoT chip EC616/EC617, and guides users to complete the development of low power applications by using related APIs

1.2 Introduction

This article briefly introduces the EC616/EC617 PMU hardware and software architecture, explains how to use and precautions of PMU related APIs. And show users how to start low-power software development on EC616/EC617 through an example.

1.3 Related nouns

1.3.1 Sleep depth

ACTIVE state：The PMU Module is off, even if there is nothing to do, the MCU is still in the loop waiting state. The power consumption is the largest.

IDLE state: At this state MCU will turn off the core working clock when there is no task, all interruption can wake up the system and restart the core clock.

SLEEP1 state: All peripherals are powered down on the basis of IDLE state, and peripheral interrupts cannot wake up the system.

SLEEP2 state: Turn off 256KB SRAM on the basis of SLEEP1 state, and keep 16KB SRAM in retention mode.

HIBERNATE state: Power off 16KB retention SRAM on the basis of SLEEP2 state

1.3.2 Voting

EC616/EC617 is a multi-tasking system. All tasks equally determines the depth of sleep. The depth of the final sleep state is affected by many factors, and finally EC616/EC617 will enter the deepest sleep mode that can be entered.

1.3.3 callback function

Function written by the user and registered to the SDK. The function registered by the user is automatically called at a certain stage in specific process of the SDK.

1.3.4 DeepSleep Timer

Software timers which support to run in deep sleep mode (SLEEP2 and HIBERNATE).

1.3.5 Retention IO

IO ports that can be maintained in specific level in deep sleep mode (SLEEP2 and HIBERNATE)

1.3.6 User NVMem

Provide users with a section of SRAM to store information that will be lost when entering SLEEP2 / HIBERNATE. The data is actually written to the Flash, but the SDK will ensure that as few writes as possible. Only when you can actually enter SLEEP2 / HIBERNATE will you actually do the write operation

1.3.7 MCU Mode

A special working state, the core frequency of the system is 16MHz, in this case, the PLL is turned off to decrease the power consumption.

2. EC616 / 617 hardware resources

2.1 PMU hardware block diagram

EC616 / 617 has 256KB + 16KB Sram, 20 Normal IO (BGA package) or 10 Normal IO (QFN). EC616 has 6 Retention IO (BGA package), EC617 has no Retention IO. Sleep depth higher than SLEEP1 (inclusive) only supports RTC wakeup and Pad wakeup, any peripheral interrupt can wake up the system under IDLE state. For EC617, low-power UART is provided, which can wake up the system directly from the serial port, and the serial port data will not be lost. In each sleep state, the power state of the module is as follows:

Table I:

	ACTIVE	IDLE	SLEEP1	SLEEP2	HIBERNATE
256KB Sram	ON	ON	ON	OFF	OFF
16KB Ret Sram	ON	ON	ON	ON	OFF
Normal IO	ON	ON	OFF	OFF	OFF
Ret IO	ON	ON	ON/OFF	ON/OFF	ON/OFF

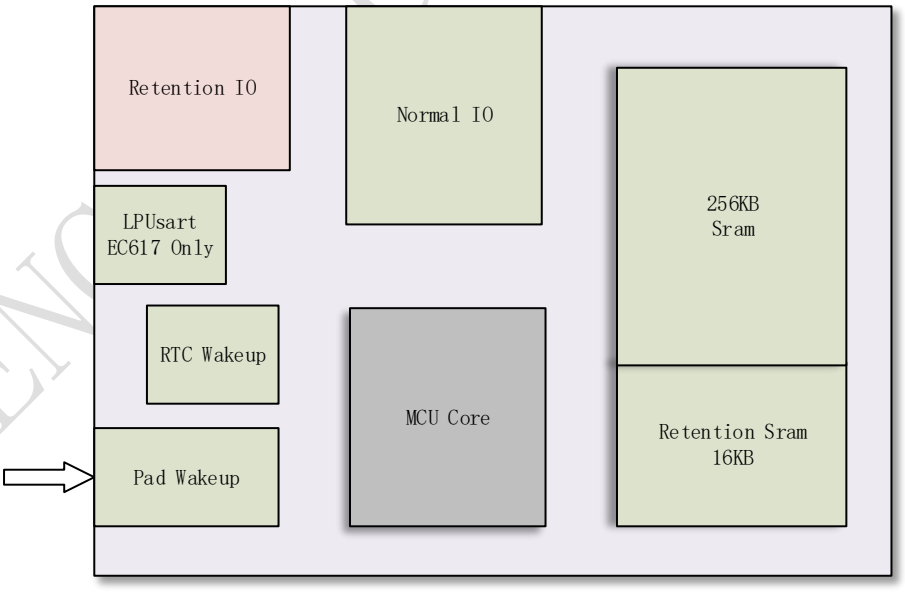


Figure 1: EC616 / 617 PMU hardware block diagram

3. EC616 / 617 PMU software

3.1 Sleep flow

3.1.1 Flow of IDLE State

When the program is idle, the software attempts to enter the idle state automatically. After awakening from the idle state, the program (PC value) starts executing from the same place before idle occur.

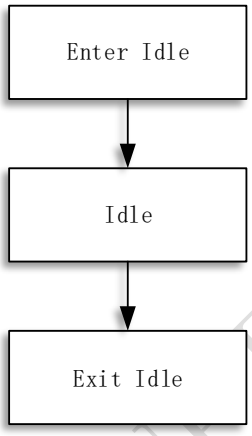


Figure 2: IDLE block diagram

3.1.2 Flow of SLEEP1 State

In Sleep1 state, due to the powered down of all peripherals. It is necessary to execute a PreSleep callback function before actually entering Sleep1, and also execute a PostSleep callback function after waking up to restore the state before sleep. Users can use the API (will be introduced in later chapters) to register the callback function by yourselves. The callback function registered by the user should be as short as possible. Do not use operations that suspend the system (such as sending and receiving operating system messages, waiting for the task queue, and calling Operating system delay OsDelay, etc.), time-consuming operations (such as flash operation, waiting for external interrupts, waiting for serial data, etc.). In order to ensure PMU efficiency and PMU work properly, the sleep process, in principle, all PreSleep callback functions and all PostSleep callback functions registered by user should not exceed 3ms. The storage and restoration of the driver-level registers are done by the SDK, which does not need users to take separate consideration.

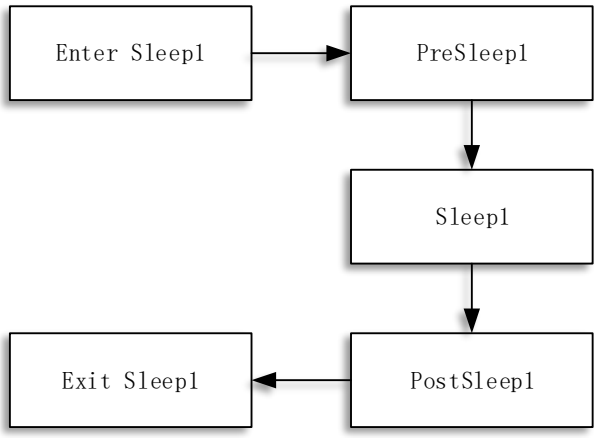


Figure 3: SLEEP1 block diagram

3.1.3 Flow of SLEEP2 State

In Sleep2 state, 256KB SRAM will be powered down, and only keep 16KB SRAM Retention. If the sleep is successful, the program will run from the beginning after the system wakes up. After completing a NB process, the user will get control in the Main_Entry function. At this time, the user can query the status through API functions (will be introduced in later chapters) to check whether the program is running for the first time after power-on or wake up from the sleep process. If the sleep fails, the program will be executed sequentially.

It is impossible to know whether sleep will succeed or fail before actually going to sleep. Therefore, users need to be prepared for the failure, that is, register the corresponding callback function to PostSleep2. Similarly, the callback function execution time of PreSleep2 and PostSleep2 needs to be strictly controlled.

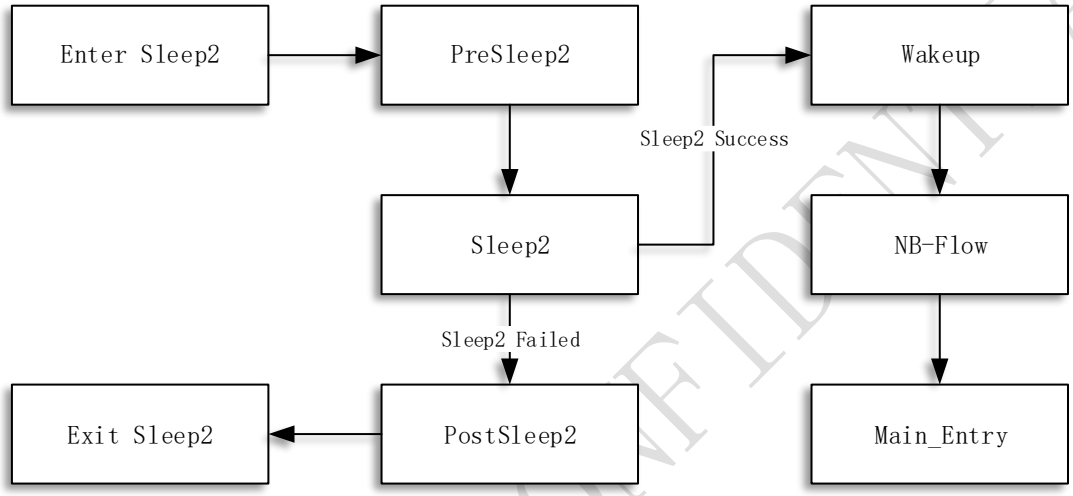


Figure 4: SLEEP2 block diagram

3.1.4 Flow of HIBERNATE State

In Hibernate state, 16KB SRAM will also be powered down. The software process is basically the same as Sleep2. Similarly, users can also register the PostHib callback function to prepare for Hibernate failure.

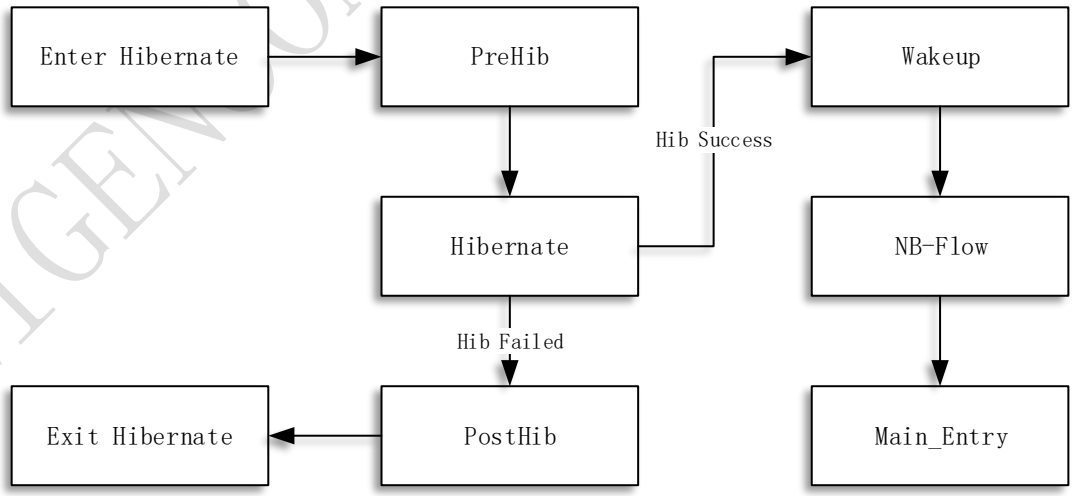


Figure 5: HIBERNATE block diagram

3.2 Sleep depth control

EC616 / 617 has four levels of sleep depth: IDLE, SLEEP1, SLEEP2, HIBERNATE

In order to achieve the lowest possible power consumption, the user should always choose the deepest sleep mode that can be entered. The method of setting the sleep depth in AT mode is as follows:

AT + ECPMUCFG = 1,4 Set the maximum sleep depth to Hibernate.

AT + ECPMUCFG = 1,3 sets the maximum sleep depth to Sleep2.

AT + ECPMUCFG = 1,2 Set the maximum sleep depth to Sleep1.

AT + ECPMUCFG = 1,1 sets the maximum sleep depth to Idle.

AT + ECPMUCFG = 0 Disable low power mode.

3.2.1 Basis for determining the application layer sleep depth

For APP layer program developers, the basis for determining the deepest sleep depth are mainly SRAM requirements and peripheral usage. The principle which users need to know is that in SLEEP1 and deeper sleep mode, peripherals are powered down. SLEEP2 has only 16KB SRAM (some of which cannot be used by users), and there is no available SRAM space in HIBERNATE state. Examples are as follows:

Table II:

	IDLE	SLEEP1	SLEEP2	HIBERNATE
Wait for USART Interrupt	YES	NO	NO	NO
Sending SPI data	YES	NO	NO	NO
Send Usart data Intermittently	YES	YES	YES/NO	YES/NO
Need 200KB SRAM	YES	YES	NO	NO
NEED 3KB SRAM	YES	YES	YES	NO

3.2.2 APP layer sleep depth control

As a low-power NB-MCU, EC616 / 617 depends on many factors to determine the depth of sleep. Users can vote to enter a certain level of sleep depth through API function, and ultimately which sleep depth EC616 / 617 enters depends on the NB underlying program. Users vote through the API to determine the maximum sleep depth allowed by the application layer. The SDK will consider and ensure that the next sleep depth is no higher than the user-specified depth. The mechanism of sleep voting is as follows:

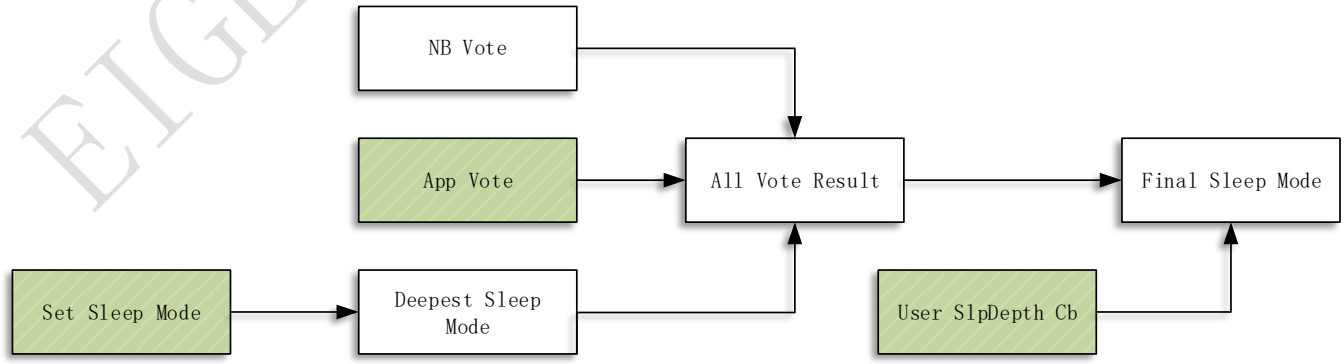


Figure 6: Voting decision tree

As shown in the figure above, the green part is the control point open to the user to control the depth of sleep, corresponding to

different API functions.

App Vote: The user controls the sleep depth by applying a voting handle (Handle) and voting to the handle

Set Sleep Mode: Directly control the maximum sleep depth. In some application scenarios, the maximum sleep depth can be lowered as we don't care about power consumption or easy programming. In addition, the maximum sleep depth can be controlled during debugging to facilitate testing.

User SlpDepth Cb: Registered user's sleep depth callback function, this function is called before actually entering the sleep process. For example, in the two-chip solution, the master controller can control the NB not to enter the sleep mode by pulling a high level of EC616 / 617 IO PIN.

The corresponding API functions for the above three control points are as follows. Here, only the function names and application occasions are briefly introduced. For specific description documents, please see doxgen or slpman_ec616 / 617.h.

（1） App Vote API

slpManRet_t slpManApplyPlatVoteHandle(const char* name, uint8_t *handle)
Apply for a voting handle and record the handle name. The handle name will be saved (maximum length is 9 bytes), and then the handle can be queried according to the handle name. The user should guarantee the uniqueness of the handle name. You can know the function running status according to the return value of the function. See the appendix for the return value and error code.
slpManRet_t slpManGivebackPlatVoteHandle(uint8_t handle)
Return a voting handle back to the system. After the return, all information of the handle will be cleared from the system. The user should clear the vote on the handle before the handle is destroyed. That is, if the slpManPlatVoteDisableSleep function is called once for a handle, the slpManPlatVoteEnableSleep must be called before destruction to ensure the matching of votes.
slpManRet_t slpManFindPlatVoteHandle(const char* name, uint8_t *handle)
The corresponding handle can be retrieved based on the name. Please ensure the uniqueness of the name when applying for a handle
slpManRet_t slpManPlatVoteDisableSleep(uint8_t handle, slpManSlpState_t status)

For a certain handle, to cancel a certain depth of sleep and above. E.g:
slpManPlatVoteDisableSleep (handle1, SLP_SLP2_STATE); // Sleep2 is prohibited, that is, you can only enter IDLE, SLEEP1
slpManPlatVoteDisableSleep (handle2, SLP_HIB_STATE); // Disable HIBERNATE, which means you can only enter IDLE, SLEEP1, SLEEP2

slpManRet_t slpManPlatVoteEnableSleep(uint8_t handle, slpManSlpState_t status)

For a certain handle, enable a certain depth of sleep. E.g:
slpManPlatVoteEnableSleep (handle1, SLP_SLP2_STATE); // Cancel the previously prohibited operation of Sleep2
slpManPlatVoteEnableSleep (handle2, SLP_HIB_STATE); // Cancel the previously prohibited operation of HIBERNATE

Pay Attention:

1. After applying for a handle, you can only vote for the same depth of sleep on the same handle. For example, only slpManPlatVoteDisableSleep (handle1, SLP_SLP2_STATE), or slpManPlatVoteEnableSleep (handle1, SLP_SLP2_STATE).

After executing slpManPlatVoteDisableSleep (handle1, SLP_SLP2_STATE),

can not execute slpManPlatVoteEnableSleep (handle1, SLP_HIB_STATE)

2. The vote for the handle can be called in multiple threads, and the vote count is introduced. After **n** times slpManPlatVoteDisableSleep function, calling **n** times of slpManPlatVoteEnableSleep function to really clear the vote.

3. It is **strongly recommended** to check the return value to ensure that each operation is successful, that is, the return value is RET_TRUE. The enumerated type corresponding to the return type slpManRet_t can be found in slpman_ec616 / 617.h. If the return value is not checked and the result is that the vote is successful, the actual vote fails, and it will be difficult to locate the problem.

Supplementary introduction, how to understand the function name:

1.Disable SLP_SLP2_STATE is understood as adding a lock before the SLEEP2 state. When the PMU encounters the SLEEP2 lock while trying to sleep, it cannot enter a deeper sleep state.

2.Enable SLP_SLP2_STATE is understood as canceling a lock in the SLEEP2 state, whether you can enter HIBERNATE later depends on whether you add a lock in HIBERNATE, if there is no lock, you can successfully enter

3. Disable SLP_SLP2_STATE twice is understood as adding two locks to SLEEP2. Therefore, only Enable SLP_SLP2_STATE twice, that is, removing two locks can really clean up the locks on SLEEP2

4. If SLP2 is not locked, go to Enable SLP_SLP2_STATE, the result is that there is no lock that can be removed, and RET_VOTE_SLP_UNDERFLOW is returned. The error given in this case is only a warning to the user, and there is no problem with enabling SLP2 sleep

（2） Set Sleep Mode API

void slpManSetPmuSleepMode(bool pmu_enable, slpManSlpState_t mode, bool save2flash)
Set the PMU enable, the maximum sleep depth. And set whether to write this configuration into flash. After the system enters hibernate or sleep2 and wakes up, the variables used by the system will be lost. If the application layer does not want to configure it again, you can choose to write this configuration to flash when setting the sleep depth for the first time. The SDK will be responsible for reading out the configuration during the system initialization and setting the settings to take effect. In order to reduce the number of flash writes as much as possible, in OPEN-MCU mode, we suggest not to save the settings to flash, because after the system wakes up, it is easy to call the function again and set the PMU to enable and control the sleep depth.

（3） User SlpDepth Cb API

void slpManRegisterUsrSlpDepthCb(pmuUserdefSleepCb_Func callback)
The callback function registered here will be called before finally going to sleep. This must be no time-consuming operations, no suspend the system or no wait for messages in the callback function. For most sleep control, voting decisions are used, but for situations where it is not convenient to participate in voting, you can use the method of registering callbacks to achieve sleep control. For example, in a system, when a pin of NB is high, the system wakes up. When this pin is low, it will enter sleep. At this time, you can register a callback function, read the pin level in the function, and determine the sleep depth according to the pin level.

3.3 Callback function before and after sleep

After entering different sleep depth, some SRAM are powered down and some peripherals are powered down. Therefore, there should have pre-sleep preparation process (PreSleep) and a post-wake recovery process (PostSleep) before and after sleep. This process is

registered in the system through the callback registration API. As mentioned earlier, the API must not be a time-consuming operation, suspend the system, or wait for semaphores. In principle, the execution time of all callback functions registered by the user should be as fast as possible.

There are two types of callback functions: Predefined and Userdefined. The callback function of the Predefined type occupies a fixed position in the callback list, and the position is determined after compilation. The position of the callback function of Userdefined type is determined by the order of registration. Therefore, it is recommended to put the operations that are sensitive to the calling order in the Predefined type, and those that are not sensitive to the calling order in the Userdefined type. At present, all peripheral drivers use the Predefined type, and the sleep recovery of the drivers has been implemented in the our SDK, the user does not need to write it by yourself. See slpman_ec616 / 617.h enumeration type: slpCbModule_t, the enumeration type is defined as follows:

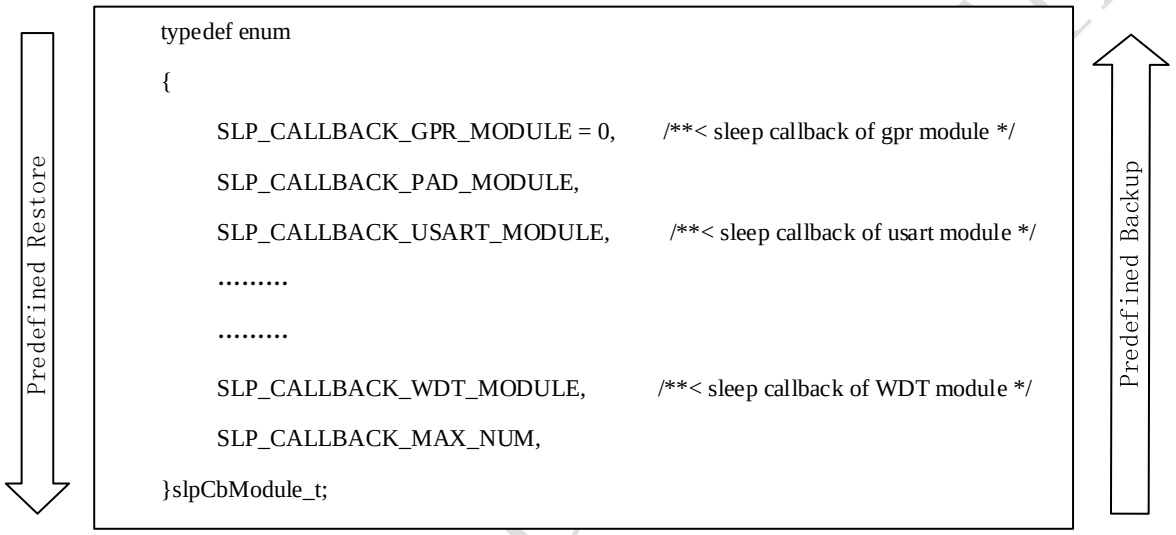


Figure 7: Callback Execution

The execution of the PreSleep process is from bottom to top, that is, the WDT module is saved first, and the GPR module is saved last.

The PostSleep process is executed from top to bottom, that is, restore to the GPR module first, and finally restore the WDT module.

For the callback function of the Userdefined type, since the program is dynamically registered, the order of registration is determined according to the order in which the program runs. The callback function written by the user is recommended to be placed in the Userdefined type. For the Userdefined type, the PreSleep and PostSleep processes will be called in order according to the order of registration.

The API functions are as follows. Only the function names and application occasions are briefly introduced here. For specific description documents, please see doxgen or slpman_ec616 / 617.h.

（1） Predefined callback

slpManRet_t slpManRegisterPredefinedBackupCb(slpCbModule_t module, slpManBackupCb_t backup_cb, void *pdata, slpManLpState state)
Register the PreSleep callback function of type Predefined. Among them, the state parameter determines the sleep mode in which the callback function is called; the pdata parameter is the input parameter when the callback function is called.
slpManRet_t slpManRegisterPredefinedRestoreCb(slpCbModule_t module, slpManRestoreCb_t restore_cb, void *pdata, slpManLpState state)

Register the PostSleep callback function of type Predefined. Among them, the state parameter determines the sleep mode in which the callback function is called; the pdata parameter is the input parameter when the callback function is called.
slpManRet_t slpManUnregisterPredefinedBackupCb(slpCbModule_t module)
Cancel the PreSleep callback function of a module
slpManRet_t slpManUnregisterPredefinedRestoreCb(slpCbModule_t module)
Cancel the PostSleep callback function of a module

（2） Userdefined callback

slpManRet_t slpManRegisterUsrdefinedBackupCb(slpManBackupCb_t backup_cb, void *pdata, slpManLpState state)
Register the PreSleep callback function of the Usrdefined type. Among them, the state parameter determines the sleep mode in which the callback function is called; the pdata parameter is the input parameter when the callback function is called.
slpManRet_t slpManRegisterUsrdefinedRestoreCb(slpManRestoreCb_t restore_cb, void *pdata, slpManLpState state)
Register the PostSleep callback function of Usrdefined type. Among them, the state parameter determines the sleep mode in which the callback function is called; the pdata parameter is the input parameter when the callback function is called.
slpManRet_t slpManUnregisterUsrdefinedBackupCb(slpManBackupCb_t backup_cb)
Cancel the PreSleep callback function of a module
slpManRet_t slpManUnregisterUsrdefinedRestoreCb(slpManRestoreCb_t restore_cb)
Cancel the PostSleep callback function of a module

（3） Callback execution

The callback function is called and executed by the SDK after registration, and the user does not need to call the callback execution function.

uint8_t slpManExcutePredefinedBackupCb(slpManLpState state)
Execute PreSleep callback function of type Predefined
uint8_t slpManExcutePredefinedRestoreCb(slpManLpState state)
Execute PostSleep callback function of type Predefined
uint8_t slpManExcuteUsrdefinedBackupCb(slpManLpState state)
Execute the PreSleep callback function of Userdefined type
uint8_t slpManExcuteUsrdefinedRestoreCb(slpManLpState state)
Execute PostSleep callback function of Userdefined type

3.4 DeepSleep Timer

Except for the SLEEP1 and IDLE states, there is no SRAM in deeper sleep states (SLEEP2 and HIBERNATE). It will cause the

OsTimer of the RTOS to be lost after a successful sleep. Therefore, if you want to maintain the operation of all OsTimer, you need to control the system can not enter a sleep depth deeper than SLEEP2. The SDK provides 4 DeepSleep Timers that can continue to run after entering the SLEEP2 or HIBERNATE state.

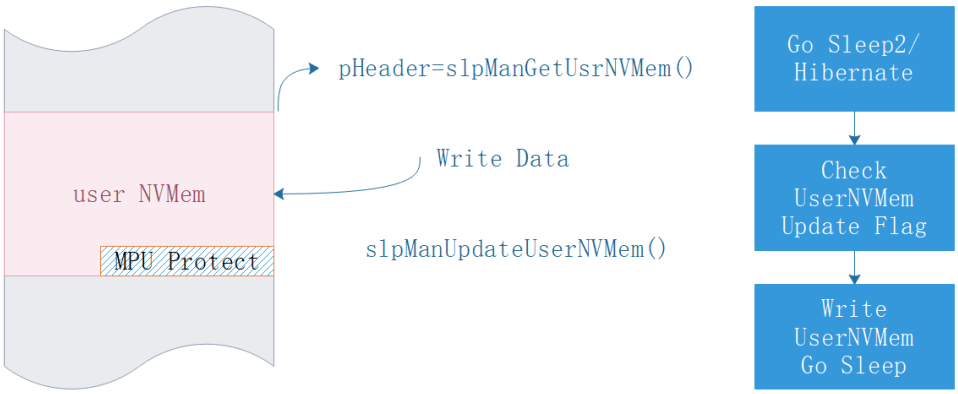
DeepSleep Timer API 如下：

void slpManDeepSlpTimerRegisterExpCb(slpManUsrDeepSlpTimerCb_Func cb)
Register the timeout callback function of DeepSleep Timer. The SDK will call this callback function after the DeepSleep Timer times out. All User DeepSleep Timer use the same callback function, you can get the ID of DeepSleep Timer from the parameters input to the callback function.
typedef void (*slpManUsrDeepSlpTimerCb_Func)(uint8_t id);
Type define of User DeepSleep Timer callback function. The input parameter of the callback function is timer id, which indicates the timer id when the timeout occurs.
void slpManDeepSlpTimerStart(uint8_t timerId, uint32_t nMs)
Start a DeepSleep Timer. Set the timerID and timeout value, the maximum timeout value is 580 hours, the unit is 1ms. If you create a DeepSleep Timer greater than 580 hours, the Timer will time out at 580 hours
bool slpManDeepSlpTimerIsRunning(uint8_t timerId)
Query whether the DeepSleep Timer is running according to the timerId
void slpManDeepSlpTimerDel(uint8_t timerId)
Delete DeepSleep Timer according to a timerId

Note: For DeepSleep Timer does not provide Timer pause function. If you want to pause the Timer, you can call slpManDeepSlpTimerDel to delete the Timer, and call slpManDeepSlpTimerStart to restart the Timer when you need to restart the Timer. It is recommended to control the frequency of use of the DeepSleep Timer. Frequent use will cause the flash life to be reduced, because every time this timer is created, a flash write will be done.

3.5 User NVMem

Users often need to save some data to flash before sleeping, to realize the recovery of the software process after waking up from deep sleep next time. Therefore, a piece of SRAM is reserved for the user, and the data written to this SRAM area will be written into the flash by the SDK before entering deep sleep, and will be restored from the flash to SRAM after wake-up. Users can think that the data in this area can be maintained all the time, but it should be noted that this area should be updated as less as possible to reduce the frequency of flash erasure. The principle of implementation is as follows:



(1) User NVMem is located in a certain space in SRAM, the user obtains the first address of this space through the slpManGetUsrNVMem function

(2) After obtaining the first address, the user can directly operate this address, and the available space is 2016Byte. You can directly specify pointers, arrays, structures, etc. to this space. To prevent cross-border access, an MPU read-only protection is made for the last 32 Bytes in this area, so the actual accessible space is 2016 Bytes.

(3) After the user writes data to this space, call slpManUpdateUserNVMem to update the write flag.

(4) When the SDK finally judges that it can enter Sleep2 or Hibernate, query whether it is necessary to write this user data area to Flash.

(5) After the system wakes up from deep sleep, the SDK will restore the data in this area from the Flash during the initialization phase.

(6) CTWING and OC platforms currently use this area for data backup.

Related APIs are as follows:

void slpManUpdateUserNVMem(void);
Tell the SDK that the content in the user data area has just been updated.
uint8_t * slpManGetUsrNVMem(void);
Obtain the first address pointer of the user data area. The user can use the (2048-32) byte length space. The variables that the user needs to keep in deep sleep can be written directly into this area. After writing, be sure to call the slpManUpdateUserNVMem function to inform the SDK that there is an update in this area, so that you can write before deep sleep.
void slpManFlushUsrNVMem(void);
Immediately write the content in this user data area to Flash instead of waiting for it to update to Flash before sleeping. It is recommended that users use this function as less as possible. The automatic writing by the SDK before deep sleep will greatly reduce the frequency of Flash writing.

3.6 Other API

3.6.1 AON IO Settings

AON IO is only available on EC616. EC617 does not have AONIO.

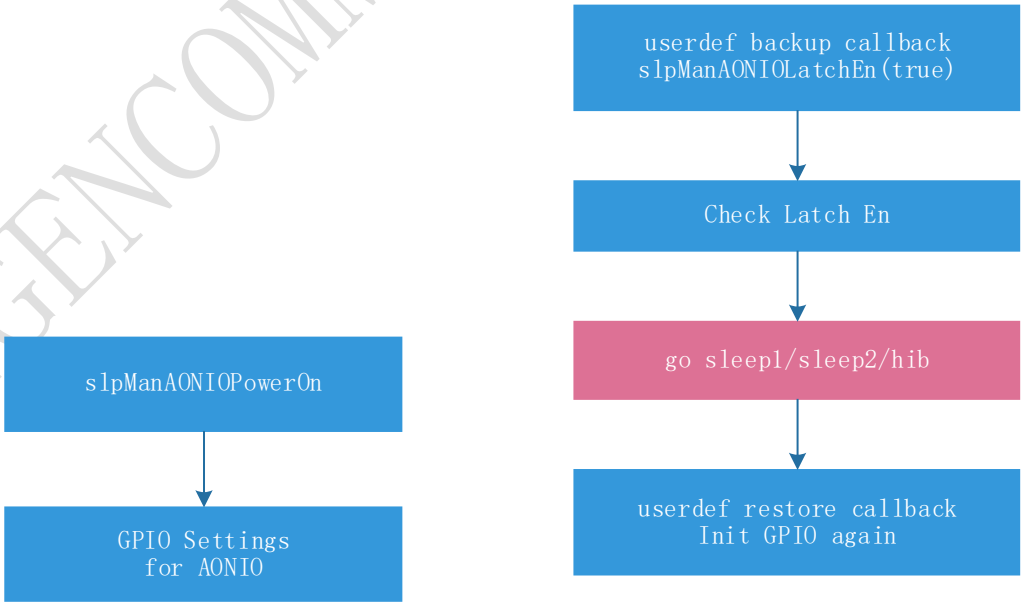
For the BGA packaged EC616, it has 6 Retention IO, which can maintain the IO level before sleep after entering sleep. These 6 GPIOs

can be configured using the GPIO driver in the Active state. However, it should be noted that the configuration of AON IO needs to be powered on, otherwise the configuration of GPIO will be invalid because AON IO is not powered. Related configuration APIs are as follows:

void slpManAONIOLatchEn (IOLatchEn en)
Set to enable AON IO. Only when enabled, Retention IO will maintain the level state during sleep, otherwise Retention IO will be powered off. Retention IO level is set using GPIO API driver. Latch only occurs immediately before sleep. If the AON IO level changes after calling this function, the pin level before sleep will latch
IOLatchEn slpManAONIOGetLatchCfg (void)
Get the enabled state of AON IO, that is, read the configuration result of slpManAONIOLatchEn
void slpManAONIOPowerOn(void)
By default, AON IO is not powered. Call this function to power on AON IO, and then use GPIO driver to operate AON IO after power on.
void slpManAONIOPowerOff (void)
Power off all AON IO. After power off, all 6 AON IO will be powered off. Then, it can not latch in sleep
void slpManAONIORelease(void)
If Latch AON IO is configured before sleep, GPIO should be configured as soon as possible after the system wakes up and the Latch of AON IO should be removed. Normally, the user should complete the Latch release within Bsp_CustomInit.

3.6.2 How to use AON IO

The configuration and initialization process of AONIO is shown on the left.



How AONIO operation in SDK before and after sleep is shown on the right

1. AONIO LDO is not open when power on / reset chip，when we need to use AONIO first we should power on
2. Then use the same API for GPIO settings to configure AONIO.

3. AONIO is not latched immediately when calling AONIOLatchEn. It take effect just before sleep enter. On the other side, you can still change the GPIO level after calling slpManAONIOLatchEn(true).

4. You can call slpManAONIOLatchEn(true) in presleep callback of sleep1/sleep2/hibernate or some other place you need.

5. SDK will check whether we need to latch AONIO after calling the presleep callback.

6. When wakeup from sleep1, UserdefRestoreCallback will execute. You need re-initialize GPIO settings for specific AONIO. There is no need to re-power on AONIO LDO.

7. After calling UserdefRestoreCallback, AONIO Release will execute by SDK. Therefore, make sure to re-configure GPIO in UserdefRestoreCallback, otherwise, AONIO Pad level will drop down.

8. For sleep2/hibernate, UserdefRestoreCallback will execute when sleep failed. As sleep failed, GPIO settings will not lost, therefore, there is no need to re-configure GPIO

9. After wakeup from sleep2/hibernate slpManAONIORelease will call after BSP_CustomInit, so we should re-initialize GPIO in BSP_CustomInit

10. Level on AONIO is the same as the configure in GPIO settings after slpManAONIORelease

11. slpManAONIORelease will happen at,

（1）after BSP_CustomInit

（2）when sleep failed

（3）when wakeup from sleep1

To keep AONIO latch during sleep，we suggest:

1. AONIO configure in BSP_CustomInit

2. register UserdefRestoreCallback for sleep1 and configure GPIO settings in that

3.6.3 To get last sleep state

After the EC616 / 617 system wakes up, the previous sleep state can be obtained through the API. Since SLEEP2 and HIBERNATE wake up, the system will start running from the beginning, so getting the previous sleep state becomes particularly important. In the user main_entry function for the SLEEP2 and HIBERNATE cases, users can customize different processes to achieve different wake-up processing. To get the last sleep state through the following API:

slpManSlpState_t slpManGetLastSlpState(void)
Get the last sleep state. The function returns the following states: SLP_SLP1_STATE, SLP_SLP2_STATE, SLP_HIB_STATE, SLP_ACTIVE_STATE Note that in the program flow, SLEEP2 and HIBERNATE will restart to run the main_entry function. And the program will be executed from the place before sleep after the wake of SLEEP1.

3.6.4 To get last wake source

slpManWakeSrc_e slpManGetWakeupSrc(void)
The wake-up source of the last wake-up can be queried through the API. The wake-up source can be POR (indicating that it is actually reset or powered on, but has not yet entered sleep), RTC, PAD or USART(in EC617 only)

3.6.5 MISC

In addition to the above APIs, EC616 / 617 PMU SDK also provides some status query functions.

bool slpManDrvSafeToSleep(void)

Check if there is a peripheral device in the working state and cannot go to sleep. When peripherals are working, EC616 / 617 can only enter IDLE state.
void slpManGetDrvBitmap(uint32_t *bitmap)
Obtain the voting status of the peripheral driver. Through this interface, you can see which peripheral is currently working because the system cannot sleep.
void slpManGetPlatBitmap(uint32_t *slp_bitmap, uint32_t *slp2_bitmap, uint32_t *hib_bitmap)
Get the voting results of SLEEP1, SLEEP2 and HIBERNATE from the platform layer. The returned result is a bitmap. Bit 0 indicates that you can enter the current level of sleep, and bit 1 indicates that you cannot enter this level of sleep. The bit position corresponds to the handle applied by slpManApplyPlatVoteHandle. For example, if slp_bitmap = 0x02 is queried, it indicates that handle 1 has not vote to SLEEP1 at this time.
slpManRet_t slpManFindPlatVoteHandle(const char* name, uint8_t *handle)
Query the corresponding handle name according to the voting handle,
slpManSlpState_t slpManPlatGetSlpState(void)
Obtain platform layer voting results
slpManRet_t slpManDrvVoteSleep(slpDrvVoteModule_t module, slpManSlpState_t status)
Peripheral driver voting API, usually the driver in this function SDK will be responsible for calling. Users generally do not need to care.

4. APP Layer PMU Software

4.1.1 Startup flow

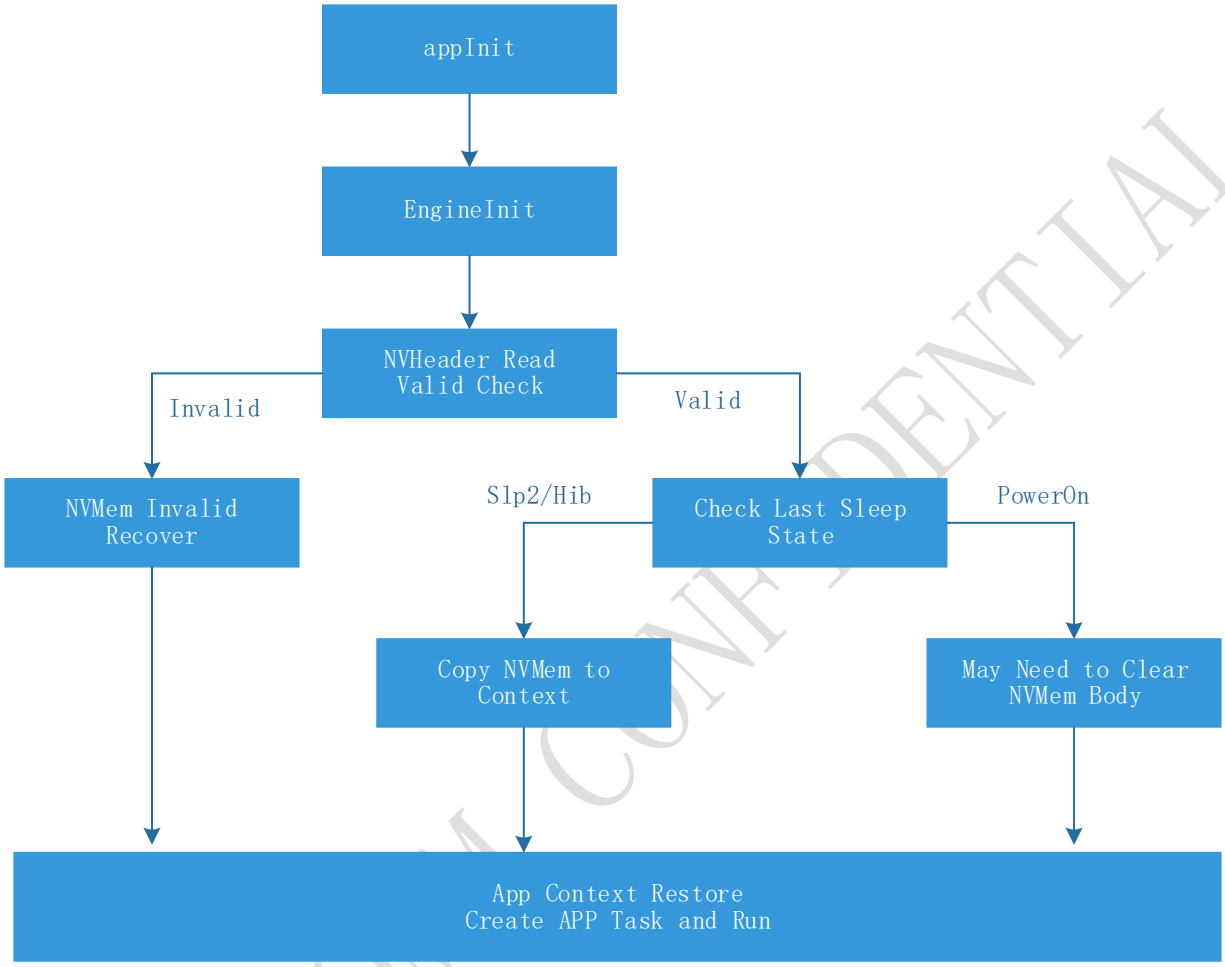
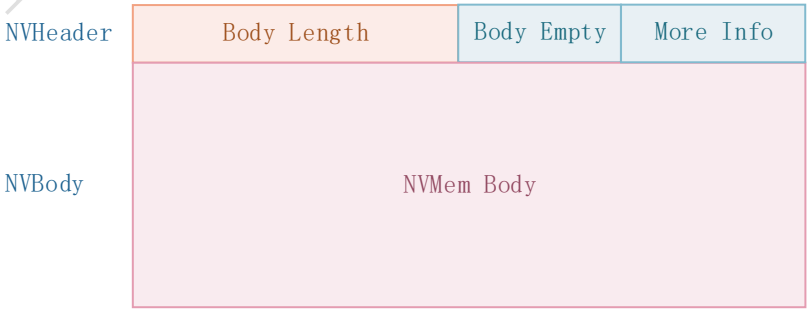


Figure 8: flowchart for Context restore when APP start up

Process description and suggestions:

1. The context restoration of low-power applications is recommended to be executed in the appInit function. Each application corresponds to a low-power entry function xxxEngineInit (), and each application corresponds to a context global variable gXXXContext and an NVM data file xxx_config.nvm.

The format of the NVM data file can be in the following format (for reference only), NVHeader contains Body Length, Body Empy Flag and other mark information



Body Length: Mark the actual length of the NV data area, so that NVMem Body can be read according to the length

Body Empty: Mark whether data has been written in NVMe Body area, if there is no data written, you don't need to do recovery or erase operation

More Info: Users can add some other information to ensure the integrity of the file according to their needs, such as MagicWord, CheckSum, Version, etc.

2. The user needs to ensure the integrity of the NVM data file. If the file is detected to be incomplete, the default value needs to be restored. There may be the following situations:

(1) First time to update a software version, NVM data is erased or the flash is empty. The file does not exist, and it needs to be initialized and written once.

(2) When the file length is changed or the storage area offset change after the program upgrade, a recovery mechanism should be considered. You can consider using checksum and other methods to check for file abnormalities and erase the file to re-initialize.

3. The purpose of using NVHeader is to reduce the possibility of reading a large amount of Flash, and only read it when it is judged that there is data in the data area. In addition, it can help achieve the integrity check of NVMem Body.

4. After completing the integrity check, check from which sleep depth the wake-up occurred. If it is the Power ON, user can decide whether to erase the file according to his needs.

5. If wake up from the Slp2 / Hib state, we first determine the Body Empty flag, if the subsequent NVMem is valid, then read and restore to ram.

6. After the context is restored, the user should create his own APP task, and then execute his own process in the APP task

7. appInit is called in a very high priority task and is used to do global initialization before the entire system. Therefore, the context initialization time of the application needs to be as short as possible. Time-consuming operations such as waiting for network connections and waiting for messages should be placed in their own application tasks.

8. In our AT Command project, we first restore the context, and then decide whether to restart the application task based on the context information. If the application task always run, the restoration of the context can also be directly put into the application task to execute. The general principle is not to block too long in appInit.。

4.1.2 Application context recovery related API

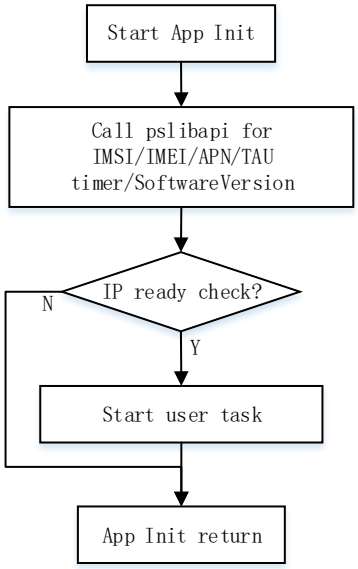
Application context information is recommended to be stored in the file system. It is easy to use and has an erasure balancing algorithm. The file system is written using the following function

```
OSAFILE OsaFopen(const char *fileName, const char* mode);
INT32 OsaFclose(OSAFILE fp);
UINT32 OsaFread(void *buf, UINT32 size, UINT32 count, OSAFILE fp);
UINT32 OsaFwrite(void *buf, UINT32 size, UINT32 count, OSAFILE fp);
INT32 OsaFseek(OSAFILE fp, INT32 offset, UINT8 seekType);
INT32 OsaFsize(OSAFILE fp);
UINT32 OsaFremove(const char *fileName);
```

4.1.3 APP PMU recovery process

If the user's application is related to the network, you need to wake up and wait for the network to get ready before initiating data services

OceanConnect PMU recovery process as an example:



5. MCU Mode

5.1.1 Mode characteristics

MCU Mode is a mode in which our chip works. In this mode:

(1) The maximum working frequency of the chip is reduced from 204MHz PLL to 16MHz RC

(2) This mode can only be transferred from the NB working state. In this mode, the NB physical layer and protocol stack do not work

(3) Only suitable for relatively simple data collection in non-networked state

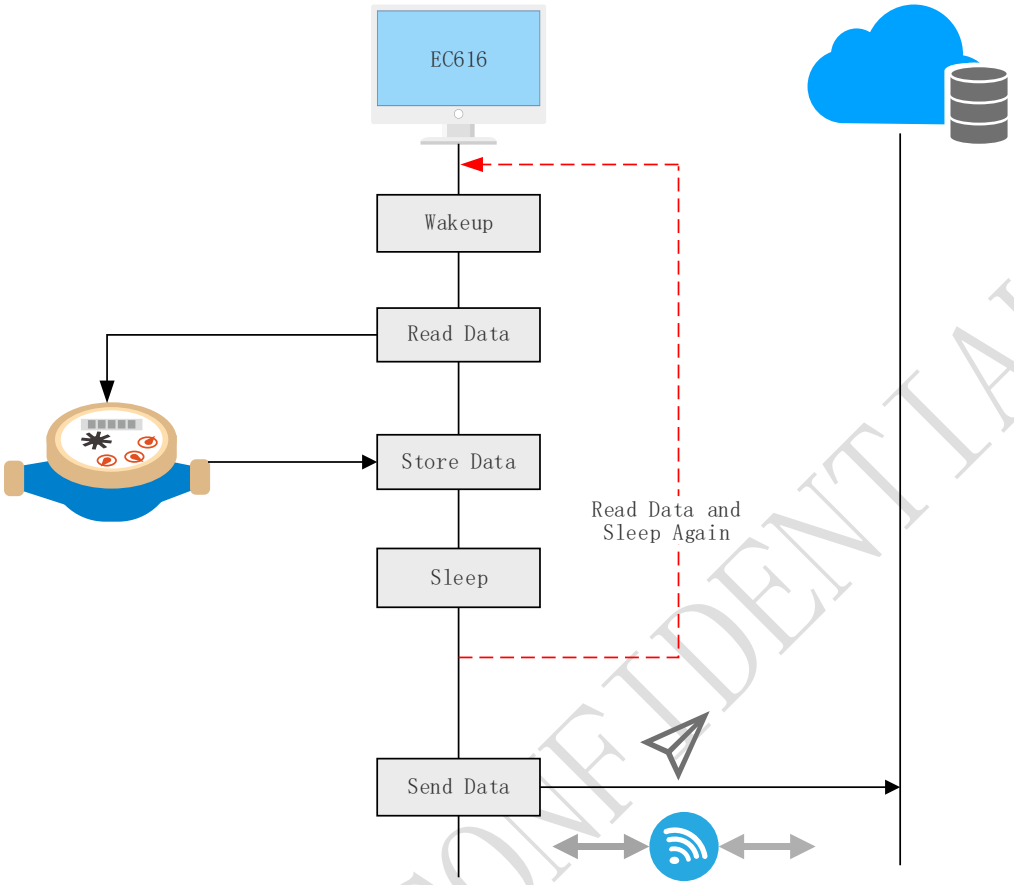
(4) The program runs in SRAM in MCU Mode, and the code and logic should be relatively simple. The collected data is also stored in SRAM, the amount of data should be small.

5.1.2 Aim of the design

Initiating the data connection between the chip and the network will bring greater power consumption. For the characteristics of NBIOT applications that are not sensitive to delay, in most cases, the data can be collected first and buffered on the device side. Send all the buffered data to the cloud at one time when needed. In this working method, the data collection work is handed over to EC616 / 617 to avoid the increase of power consumption caused by connecting to the network when collecting data.

5.1.3 Example

A common application scenarios of MCU Mode



(1) The EC616 / 617 NB network is off most of the time in a water meter data collection solution

(2) User can choose to wake up to enter the MCU Mode next time before sleeping. Configure Deep Sleep Timer to wake up into MCU Mode at the expected time before sleep

(3) The EC616 / 617 can wake up at a certain interval to collect data. The data is saved in the SRAM(16KB), or directly stored in the Flash to try to enter a deeper sleep mode. At this time, the working clock uses 16M RC

(4) In MCU Mode, you can cycle in and out of sleep, wake up, collect data and then go to sleep

(5) After the data is stored to a certain amount, wake up the system to exit MCU Mode and try to send data to the network

(6) The network side obtains a whole wave of data after a network connection

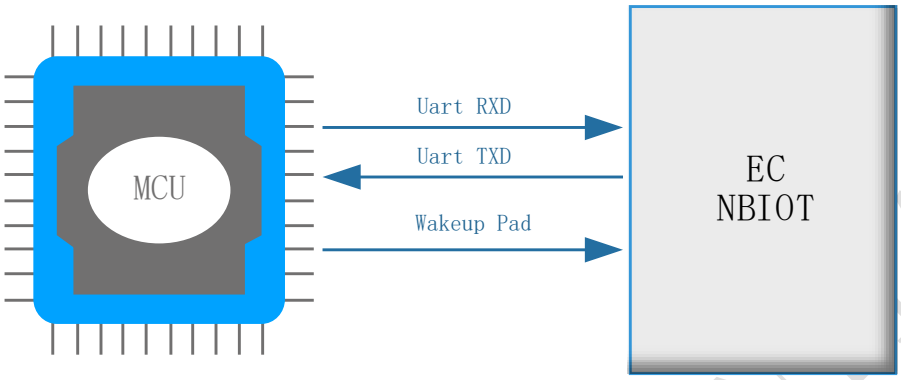
6. Wake-up from dual-chip solution

When the user program runs on the EC616, the user program can flexibly control the chip to wake up or sleep through API functions. When EC616 works as a module, it will also work in low-power mode. When an external MCU communicates with EC616, it needs to follow a wake-up protocol. Only after EC616 wakes up can data be sent. We offer two wake-up solutions, each with different characteristics:

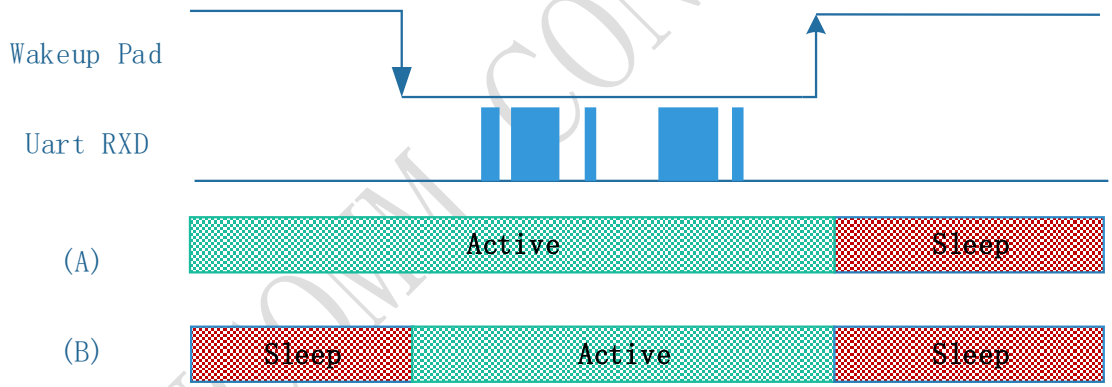
For EC617, the biggest difference from EC616 is that it has a low-power UART, so in the two-chip solution, it is not necessary to consider the wake-up of the chip, and the UART data can be sent directly at any time. Therefore, the following scheme is only applicable to EC616.

Scheme One:

The NB and the MCU are connected via Uart, and a GPIO of MCU is connected to the Wakeup Pad of the NB, which specific Pad should correspond to the code running in the NB. Under this scheme, the power consumption is the lowest. Once the command is sent and received, it can go to sleep.



When the MCU needs to send data, first pull the Wakeup Pad low. Then intermittently send "AT \ r \ n" to determine whether the NB is ready. After receiving the OK reply, it starts to send the UART command that needs to be sent. After the command is sent, if the MCU does not have subsequent commands to send, the Wakeup Pad can be pulled up (You don't need to wait until you reply before pulling high). Case A: In the Active state (non-sleep state), the NB will not go to sleep when the Wakeup Pad is low. At this time, the command from the MCU can be successfully received. Case B: In the Sleep state, when the Wakeup Pad pulls NB low to wake up, the serial port sends and receives commands, then pull the Wakeup Pad high, and the NB goes to sleep.。



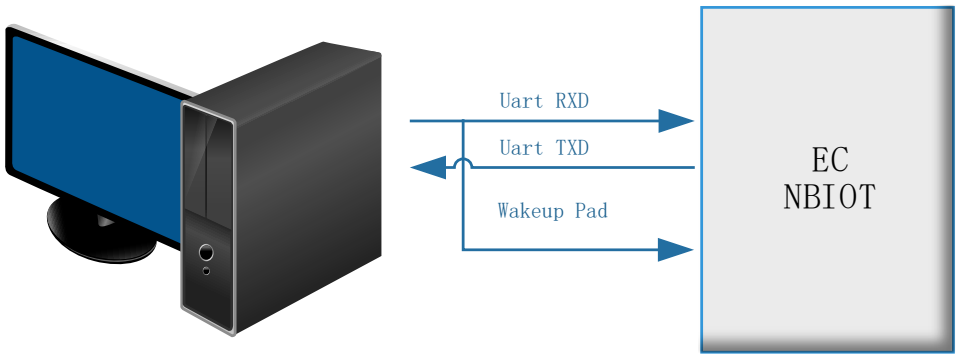
Scheme Two:

After the NB wakes up the system due to an external interrupt, it will delay a period of time before going to sleep. In this way, the user can set the delay time. The delay time after wake-up can be set through the AT command. It should be noted that in order to ensure the successful setting, it needs to be set immediately after the system is powered on, because the NB will not immediately go to sleep at this time. After setting, this configuration is written to flash immediately, and will take effect after power on again. The corresponding AT commands are as follows:

AT + ECPCFG = “slpWaitTime”, 4000

The last 4000 stands for waiting 4000ms after wake-up and then enters sleep. You can set any value according to your needs. The maximum range is 65535ms. It should be noted that the first command of each wake-up will be lost and needs to be sent again within the delay. This configuration will be written to flash, and neither sleep nor hardware reset will clear the configuration.

Under this scheme, the following wiring methods can be used:



NB is awakened by the transition level on RXD. After awakening, it starts the delay timer and waits for the expected time before enabling NB sleep. This scheme is simpler and more direct than Scheme 1, but due to the fixed delay, there will be idle time waiting for sleep after the UART data is sent and received. In addition, considering that the user may need to send and receive several commands after sending and receiving an AT command, you can call slpManStartWaitATTimer () after receiving the AT command to delay another slpWaitTimer time. For details, see app.c of at_command project in SDK. Every message received in atCmdProcessEntry will call slpManStartWaitATTimer () to start a timer and delay the sleep time by slpWaitTimer milliseconds.

```
static void atCmdProcessEntry(void *arg)
{
    uint32_t data_block_len;
    uint32_t at_last_cnt = 0;

    osMessageQueueAttr_t queue_attr;

    memset(&queue_attr, 0, sizeof(queue_attr));

    queue_attr.mq_size = sizeof(atcmd_queue_buf);
    queue_attr.mq_mem = atcmd_queue_buf;
    queue_attr.cb_mem = &atcmd_queue_cb;
    queue_attr.cb_size = sizeof(atcmd_queue_cb);


    //init message queue
    mid_atcmd_msgqueue = osMessageQueueNew(MSGQUEUE_OBJECTS, sizeof(msgqueue_obj_t), &queue_attr);

    EC_ASSERT(mid_atcmd_msgqueue, mid_atcmd_msgqueue, 0, 0);

    //wait for at command
    if(UsartAtCmdHandle != NULL)
        UsartAtCmdHandle->Receive(at_uart_recv_buf, AT_CMD_BUFFER_SIZE);

    while(1)
    {
        msgqueue_obj_t msg;
        osStatus_t status;
        ARM_USART_STATUS uart_status;
        uint32_t mask;

        //wait for message
        status = osMessageQueueGet(mid_atcmd_msgqueue, &msg, 0, osWaitForever);

        slpManStartWaitATTimer();           //Wait AT for slpWaitTime in order to receive next command

        if(status == osOK)
        {
            EC_ASSERT(msg.recv_cnt > at_last_cnt, msg.recv_cnt, at_last_cnt, 0);

            data_block_len = msg.recv_cnt - at_last_cnt;
```

By default, using scheme two will start this sleep wait timer in two situations:

(1) The system has just been powered on or awakened

(2) AT serial port received a command (whether it is a valid command or an invalid command)

The wake-up procedure for EC616 on the MCU side is as follows:

(1) Regardless of whether EC616 is awake or sleeping, the MCU should try to shake hands with EC616 before attempting to send AT commands

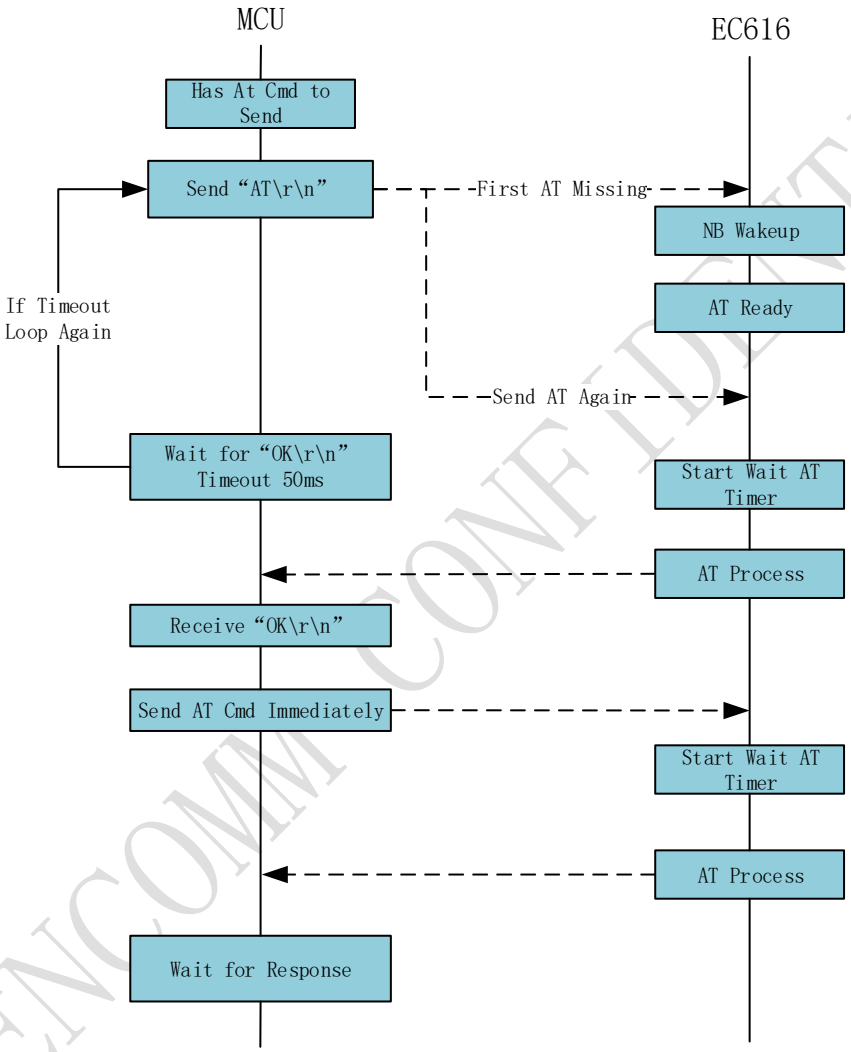
(2) The MCU sends "AT \ r \ n" as a handshake signal and waits for the reply from EC616. Received "OK \ r \ n" means successful handshake

(3) The MCU sends a handshake signal every 50ms(for example) until it receives a reply from EC616

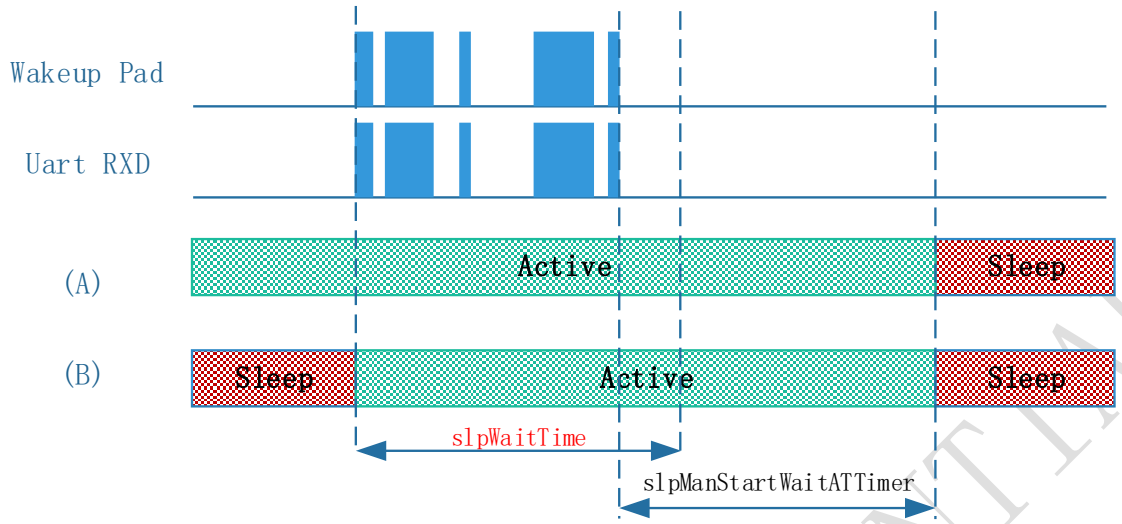
(4) After the handshake is successful, the MCU immediately sends the actual AT command that needs to be sent, and waits for a reply (the timeout at this time is the Timeout corresponding to the AT command)

(5) After the Start Wait AT Timer is executed, it will wait for slpWaitTimer milliseconds, and the next AT command needs to be sent within this time interval

(6) EC616 may go to sleep after slpWaitTimer, but whether to go to sleep or not should consider other situations (other voting results).。



Each time atCmdProcessEntry receives an AT message, it will delay the sleep time again by slpWaitTime. For example, when the original sleep delay has 2000ms remaining and the AT message is received again, the sleep delay will become slpWaitTime again.



7. Test Result

7.1 Wake-up interrupt response test

Interrupt response time of EC616 is different for different sleep depths. The NB entered the PSM mode during the following test::

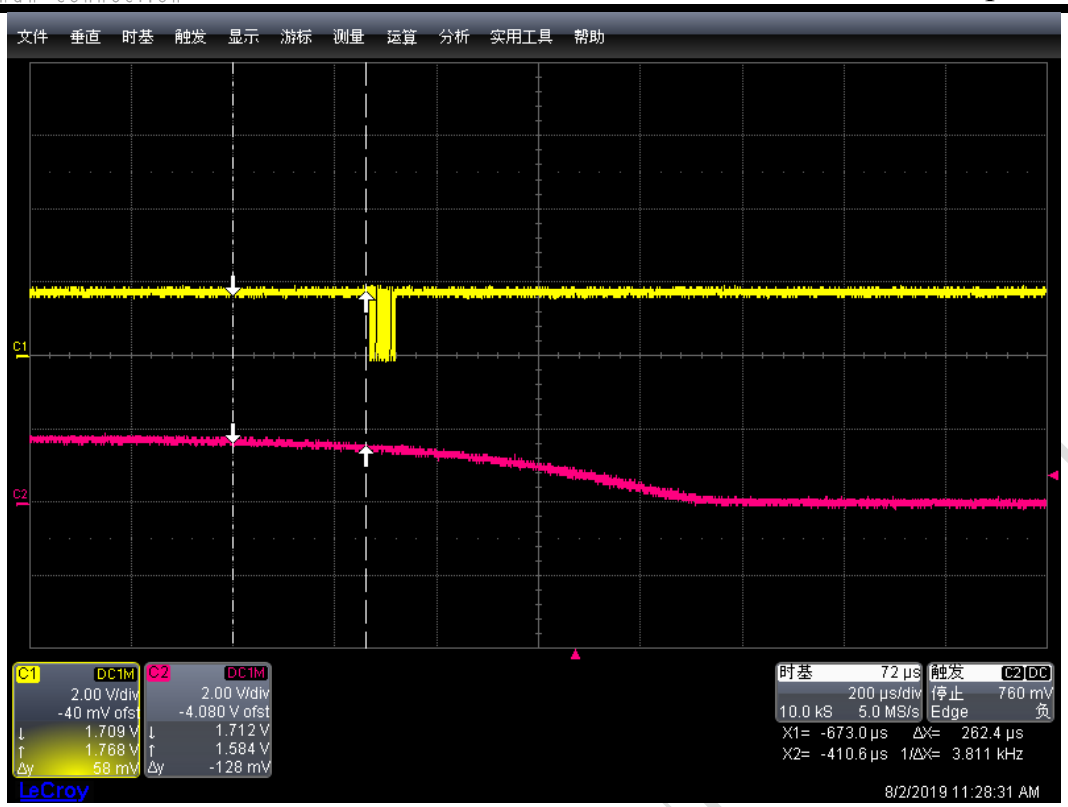
Testing method:

After entering sleep, press Wakeup0 to wake up, initialize GPIO in Pad0_WakeupIntHandler and flip the IO level many times, measure the time from the key trigger to the system wake up to the execution of the interrupt processing function. The ARM does not enable interrupts by default after booting up, users need to use NVIC_EnableIRQ (PadWakeup0_IRQn) to start Wakeup0 interrupt as soon as possible in BSP_CustomInit ().

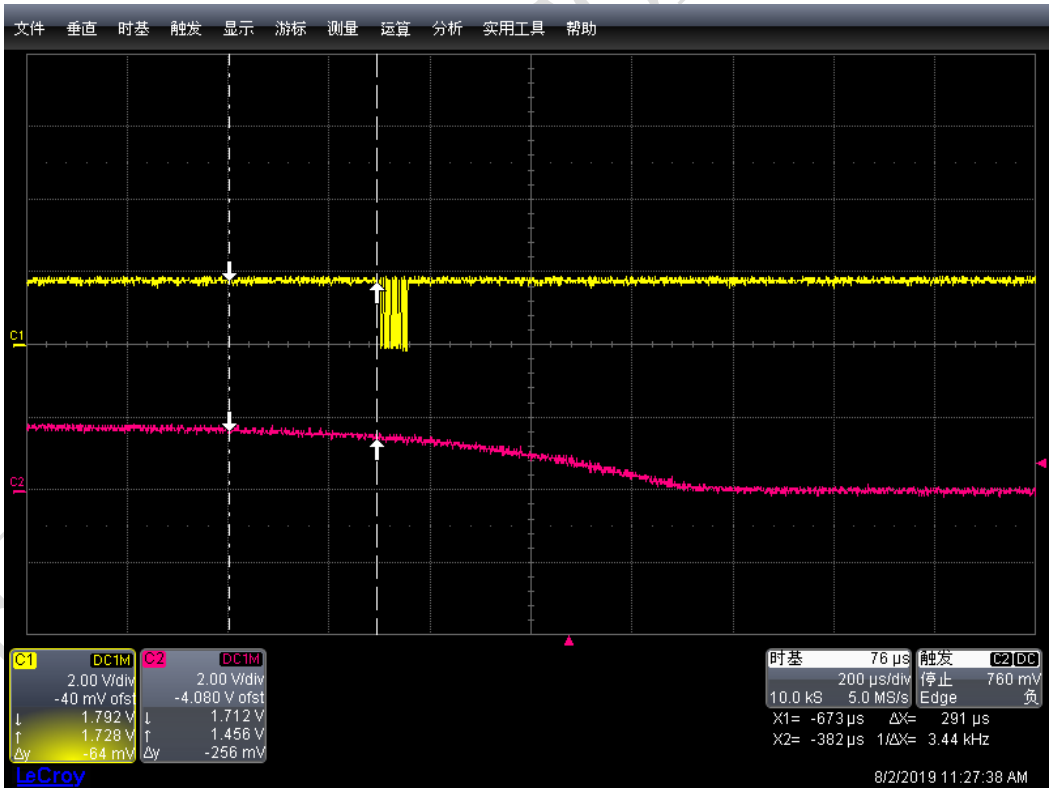
The interrupt service functions are as follows (users can implement their own service functions):

```
void Pad0_WakeupIntHandler(void)
{
    if(slpManExtIntPreProcess(PadWakeup0_IRQn)==false)
        return;

    //add custom code below//
    GPIO_Test_Init()

    for(uint8_t i;i<9;i++)
    {
        GPIO_Test_High();
        GPIO_Test_Low();
    }
```

In the Active state (without PMU enabled), the interrupt response time is about 262us:：



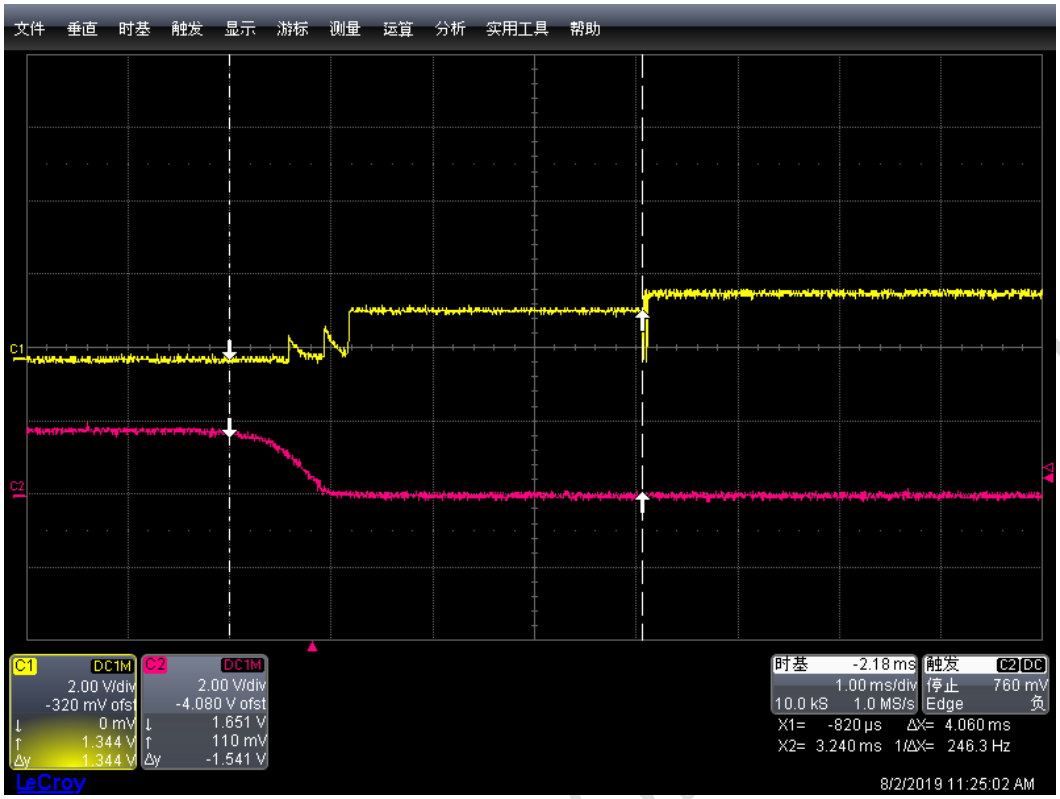
The maximum interrupt response time is about 290us in Idle state



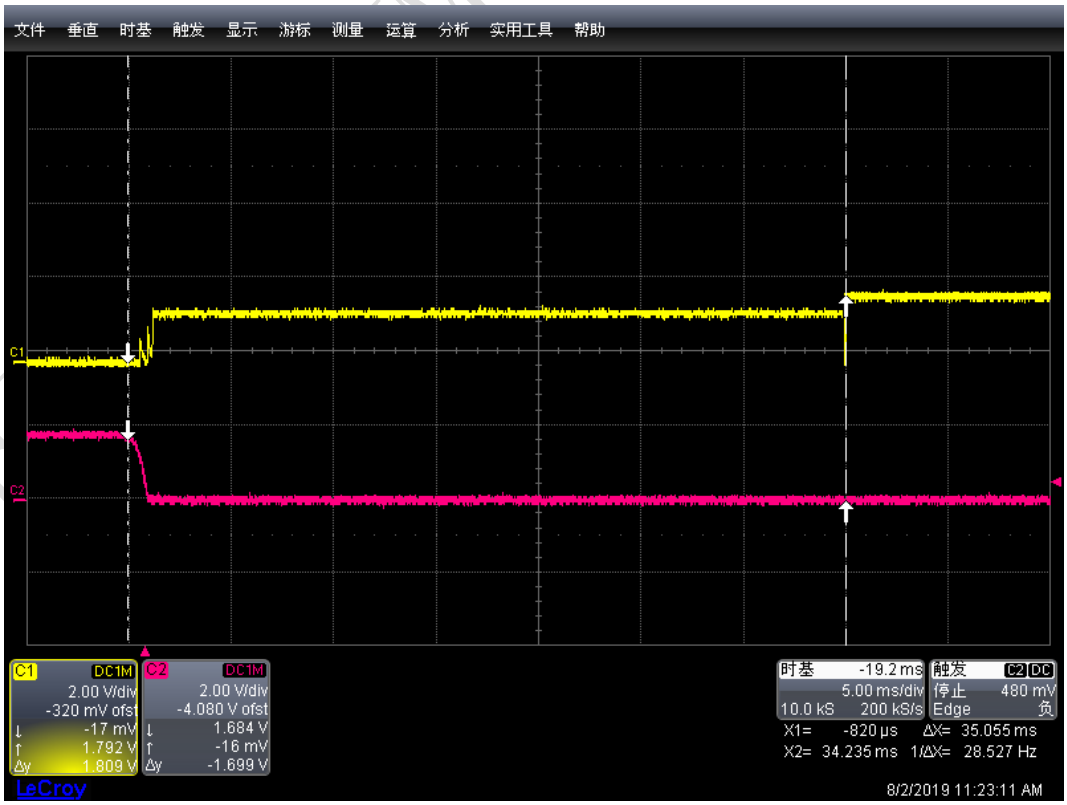
The maximum interrupt response time is about 4.1ms in Sleep1 state:

It is normal to see that the IO level jumps and does not pull to very high before interrupt execution, because the default IO is in a

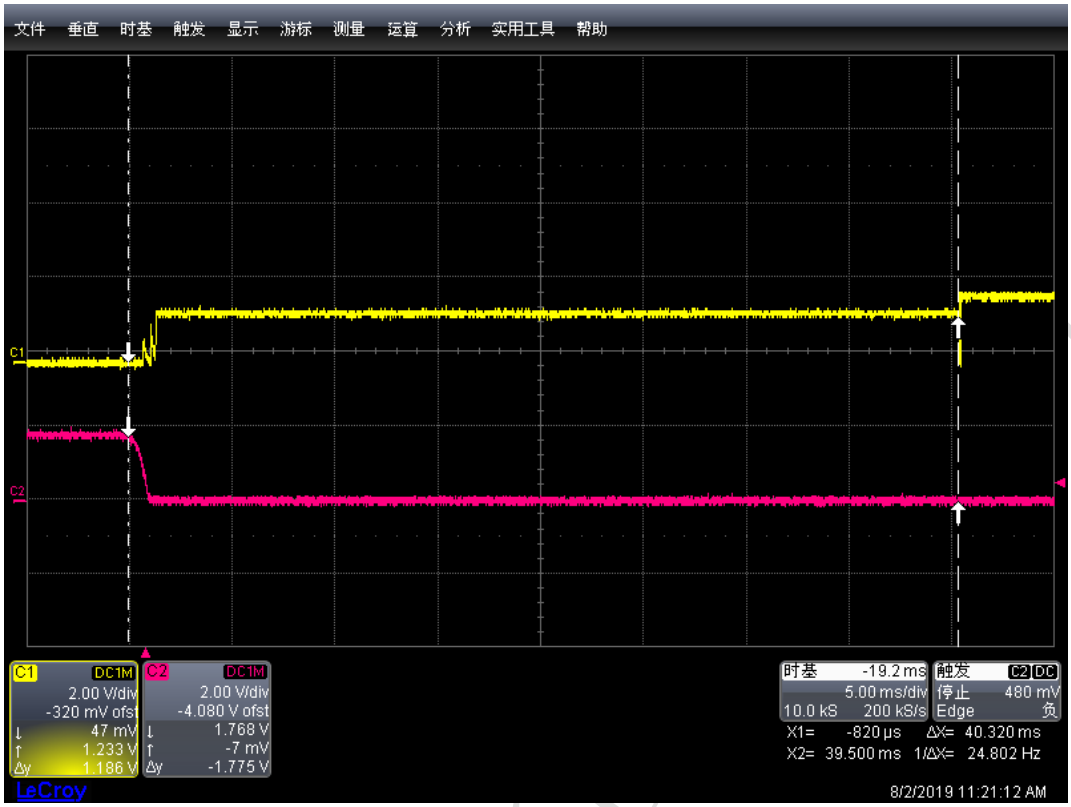
weak pull-up state after power-on. After entering the interrupt service function, IO is set to output and then a stable level can be achieved. The wake from Hibernate and Sleep2 is the same phenomenon.



The maximum interrupt response time is about 35ms in Sleep2 state:



The maximum interrupt response time is about 41ms in Hibernate state:



7.2 Dual chip wake-up scheme 2 testing

In order to explain the wake-up method of scheme two more clearly, run a python script on the PC to demonstrate this wake-up process. Through API encapsulation, it is possible to realize no sense of handshake during AT sending process.

Test method: Make sure that UART1_RXD is connected to Wakeup0, the bottom panel is connected to the PC and find the corresponding UART interface. In this example, the AT UART interface is mapped to COM12. Make sure that AT + ECPCFG = “slpWaitTime”,4000 is configured, where 4000 can be adjusted down according to the test situation, and the automated test process can usually be completed within 1000ms.

(1) Ec616_UartSend (at_str) This function is the API to send AT to the application layer.

(2) Before AT Command is officially sent, there will be a handshake process to wake up EC616. See Ec616_Uarthandshake ().

In Ec616_Uarthandshake (), the AT is sent once every interval and waits for the OK reply. Once the OK reply is received, it means that the handshake is successful.

(3) After the handshake is successful, the actual AT command needs to be sent immediately. At this time, the user can set different timeout waiting time according to different AT commands

(4) Wait for AT's reply within the timeout.

(5) Ec616_Uarthandshake () can be hidden by function encapsulation. Polling_OK () is the process of waiting for the OK character to be received, and the MCU side can have other implementation methods. So to send the AT command on the MCU side, the real call is Ec616_UartSend (at_str).

```
import serial

ser = serial.Serial('COM12',115200)

AT=b'AT\r\n'
CESQ=b'AT+CESQ\r\n'


def Polling_OK():
    for i in range(1, 10):
        data = ser.readline()
        if(data == b'OK\r\n'):
            print(data)
            return True;
        elif(data != b''):
            print(data)
    return False;

def Ec616_Uarthandshake():
    ser.timeout = 0.1
    print("EC616 handshake Start");
    for i in range(1, 10):
        ser.write(AT)
        if(Polling_OK() == True):
            print("EC616 Wakeup Complete");
            return True
    return False;

def Ec616_UartSend(at_str):

    if(Ec616_Uarthandshake()):
        ser.timeout = 10
        ser.write(at_str)

        Polling_OK();
    else:
        print('Wakeup Timeout')


def WakeupTest_Start():
    print('Start Test')
    if(ser.is_open == True):
        ser.close()
    ser.open()
    Ec616_UartSend(CESQ)
    ser.close()


if __name__ == '__main__':
    WakeupTest_Start()
```

8. Error Code

All PMU related functions for user calls will provide return values. It is recommended to check the return value after calling the function to ensure that the return value is RET_TRUE. All other situations indicate that there is a problem with the function execution. The corresponding return value is explained as follows:

```
RET_TRUE,                   /**< Error code: no error */
RET_UNKNOW_FALSE,           /**< Error code: unknow error */
RET_CB_ARRAY_FULL,          /**< Error code: callback array is full */
RET_CALLBACK_EXIST,         /**< Error code: duplicate callback in callback array */
RET_CALLBACK_UNEXIST,       /**< Error code: callback unexist */
RET_VOTE_SLP_OVERFLOW,      /**< Error code: vote overflow (above 256) */
RET_VOTE_SLP_UNDERFLOW,     /**< Error code: vote underflow (below 0) */
RET_ZERO_NAME_LEN,          /**< Error code: input name is empty */
RET_INVALID_NAME,           /**< Error code: input name has illegal character */
RET_VOTE_HANDLE_FULL,       /**< Error code: no place for more vote handle */
RET_INVALID_HANDLE,         /**< Error code: vote handle is invalid */
RET_INVALID_VOTE,           /**< Error code: the vote state is invalid */
RET_VOTE_CONFLICT,          /**< Error code: vote conflict */
RET_GIVEBACK_FAILED,        /**< Error code: vote handle give back error */
RET_HANDLE_NOT_FOUND,       /**< Error code: vote handle not found */
```

9. More common problems

9.1 API usage restrictions and solutions

As a NB MCU, EC616 / 617 must ensure the correct operation of the NB protocol stack while executing user code, so there are some restrictions in use. Only the limitations related to low power consumption are described here.

(1) The count of OSDelay is only valid in Idle and Sleep1

Because in Sleep2 and Hibernate state SRAM will be mostly powered off or completely powered off, so the RTOS system running on it can not continue to run. That is, after waking up, RTOS will restart running, so OSDelay information will be lost.

If users want to keep OSDelay effective, they should not vote for Sleep2 and Hibernate. If a vote is taken, all delays of OSDelay will not take consider.

How to start a timer that is valid in each state:

DeepSleep Timer can be maintained under Idle, Sleep1, Sleep2, Hibernate.

(2) There are only 4 DeepSleep Timers, and the maximum counting range is 580 hours

Users can specify different Timer IDs, but the number of Timers can only be 4 at the same time. It is recommended to use the DeepSleep Timer as less frequent as possible, especially to prevent the DeepSleep Timer from being enabled with a very short timeout, as it will limit continuous sleep. Frequent in and out of sleep is not conducive to reducing power consumption.

(3) Callback functions before and after sleep

The callback function before and after sleep is called immediately before sleep. At this time, the system closes the interrupt, so there is no task scheduling. Therefore, the callback function cannot use operations such as sending and receiving messages, or suspending tasks. Since the operation of all callback functions is execute in an interrupt mask state, the execution time of all callback functions registered by users must not exceed 3ms. Time-consuming operations should be execute before voting to sleep.

10. Version

Version	Date	Note
Draft	2019-03-20	wqzhao：Basic version
V1.1	2019-10-21	wqzhao：Add note for slpManGetWakeupSrc
V1.2	2019-11-28	wqzhao：Added instructions on how to understand the name of the voting function
V1.3	2020-03-25	wqzhao：Describes a more general process and suggestion for APP PMU flow
V1.4	2020-04-06	wqzhao：Add EC617
V1.5	2020-05-11	wqzhao：How to use AONIO

11. About US

Shanghai EigenCOMM Technology Co.,Ltd is established in Feb 2017 in Zhangjiang Hi-Tech park, Shanghai China (www.eigencomm.com) . EigenCOMM focuses on cellular based IoT chipset and solution. With accumulation of experience over the years in algorithm, L1/L2/L3 protocol, SoC, RF , platform & application software as well as the communication architecture and system, team makes its way to IoT industry.

The ultra-low power NB-IoT chipset EC616(S) has already enter mass production stage. The 4G Cat.1 chipset EC618 is in engineering samples stage, 5G is also in the R&D stage.

Eigencomm Technoligies Ltd.

Address：Room 707, No 298 Xiangke Road, Shanghai, China

Zip Code：201210

Phone：

E-mail：

Website：http://www.eigencomm.com

Support Window

E-Mail：support@eigencomm.com